RA1 - Ingestão e Preparação dos Dados

1. Definição do Problema e Contexto Analítico

Os sistemas de saúde enfrentam desafios contínuos relacionados à readmissão hospitalar precoce de pacientes. A readmissão não planejada em até 30 dias após a alta hospitalar eleva os custos do sistema de saúde, sobrecarrega a infraestrutura clínica e frequentemente sinaliza dificuldades na continuidade do tratamento ou complicações pós-alta.

Este projeto investiga fatores demográficos, clínicos e de utilização hospitalar associados à **readmissão em até 30 dias** entre pacientes diagnosticados com diabetes em ambiente hospitalar nos Estados Unidos.

Pergunta Analítica Central

> **Quais fatores clínicos, demográficos e de utilização hospitalar estão associados à readmissão de pacientes com diabetes em até 30 dias após a alta?**

Perguntas Analíticas Específicas

1. **Tempo de Internação:** De que forma o tempo total de permanência hospitalar (`time_in_hospital`) influencia a taxa de readmissão precoce (<30 dias)?
2. **Histórico de Utilização Prévia:** Pacientes com histórico de internações e atendimentos de emergência no ano anterior (`number_inpatient` , `number_emergency` , `number_outpatient`) apresentam probabilidade significativamente maior de readmissão?
3. **Manejo Farmacológico:** A realização de ajustes terapêuticos ou prescrição de insulina durante a internação (`change` , `diabetesMed` , `insulin`) está associada a reduções ou aumentos no risco de retorno hospitalar?
4. **Comorbidades e Diagnósticos:** Quais categorias de diagnóstico primário (`diag_1`) e contagens de diagnósticos (`number_diagnoses`) estão mais associadas ao retorno precoce?

---

2. Fontes de Dados

- **Identificação do Dataset:** *Diabetes 130-US Hospitals for Years 1999–2008*.
- **Origem e Repositórios:** Obtido via Kaggle e originalmente disponibilizado pelo UCI Machine Learning Repository.
- **Formato:** Tabular delimitado (CSV - `diabetic_data.csv`).
- **Volume Inicial:** 101.766 registros de atendimentos hospitalares e 50 atributos.
- **Unidade Amostral:** Cada registro representa um atendimento hospitalar individual (`encounter_id`). Um paciente pode aparecer em múltiplos atendimentos ao longo do período (`patient_nbr`).
- **Principais Variáveis:**
  - *Demográficas:* `age` , `gender` , `race` .

- *Utilização de Serviços:* `time_in_hospital`, `number_outpatient`, `number_emergency`, `number_inpatient`.
- *Diagnósticos:* `diag_1`, `diag_2`, `diag_3`, `number_diagnoses`.
- *Tratamento e Medicações:* `max_glu_serum`, `A1Cresult`, `metformin`, `insulin`, `change`, `diabetesMed`.
- *Desfecho (Alvo):* `readmitted` (`<30`, `>30`, `NO`).

---

3. Avaliação da Qualidade e Justificativa das Decisões de Limpeza

A exploração inicial identificou problemas de qualidade típicos de dados de saúde:

- **Codificação de Dados Ausentes:** O dataset utiliza a string `'?'` para representar valores não informados. Esses valores foram padronizados para nulos reais (`np.nan`).
- **Colunas com Esparsidade Excessiva:** A coluna `weight` possui mais de 96% de dados ausentes; `payer_code` e `medical_specialty` também apresentam percentuais proibitivos de ausência (>40%). Essas colunas foram removidas nesta etapa.
- **Inconsistências Categóricas:** A coluna `gender` continha 3 registros como `'Unknown/Invalid'`, que foram padronizados para `'Não informado'`.
- **Integridade dos Identificadores:** As colunas `encounter_id` e `patient_nbr` foram convertidas de numéricas para texto (`string`), pois representam identificadores e não grandezas quantitativas.
- **Definição da Variável Alvo:** Foi criado o indicador binário `readmitido_30_dias`, atribuindo valor `1` para retorno em menos de 30 dias (`<30`) e `0` para retorno após 30 dias (`>30`) ou ausência de readmissão (`NO`).
- **Escopo do RA1:** Não são realizadas transformações analíticas antecipadas, agregações, indicadores ou gráficos de conclusão, respeitando estritamente os critérios da disciplina. O conjunto final preparado é validado por asserções e exportado em disco.

In [1]:
```
# 1. Ingestão dos Dados a partir do Kaggle
import kagglehub
from kagglehub import KaggleDatasetAdapter
import pandas as pd
import numpy as np
from IPython.display import display
from pathlib import Path

file_path = "diabetic_data.csv"

# Carregamento reproduzível do dataset a partir do Kaggle
df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "razanihababdellatif/diabetes-hospital-readmission-dataset",
    file_path,
    pandas_kwargs={'keep_default_na': False},
)
df_original = df.copy()

print(f"Dataset carregado com sucesso via KaggleHub!")
print(f"Dimensões originais: {df.shape[0]} atendimentos hospitalares e
{df.shape[1]} atributos.")
display(df.head())
```

```
C:\Users\HelenLauren\AppData\Local\Programs\Python\Python310\lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Dataset carregado com sucesso via KaggleHub!
Dimensões originais: 101766 atendimentos hospitalares e 50 atributos.
```

Columns 1–8 of 22

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id
0	2278392	8222157	Caucasian	Female	[0-10)	?	6
1	149190	55629189	Caucasian	Female	[10-20)	?	1
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1
3	500364	82442376	Caucasian	Male	[30-40)	?	1
4	16680	42519267	Caucasian	Male	[40-50)	?	1

Columns 9–15 of 22 · first column repeated

	discharge_disposition_id	admission_source_id	time_in_hospital	...	citoglipton	insulin	glyburide-metformin
0	25	1	1	...	No	No	No
1	1	7	3	...	No	Up	No

	discharge_disposition_id	admission_source_id	time_in_hospital	...	citoglipton	insulin	glyburide-metformin
2	1	7	2	...	No	No	No
3	1	7	2	...	No	Up	No
4	1	7	1	...	No	Steady	No

Columns 16–22 of 22 · first column repeated

	glipizide-metformin	glimepiride-pioglitazone	metformin-rosiglitazone	metformin-pioglitazone	change	diabetesMed	readmitted
0	No	No	No	No	No	No	NO
1	No	No	No	No	Ch	Yes	>30
2	No	No	No	No	No	Yes	NO
3	No	No	No	No	Ch	Yes	NO
4	No	No	No	No	Ch	Yes	NO

5 rows × 50 columns

In [2]:
```
# 2. Compreensão e Avaliação da Qualidade dos Dados
print("--- Informações Estruturais do Dataset ---")
df.info()

# O dataset utiliza '?' para representar dados faltantes. Padronização para NaN:
df.replace('?', np.nan, inplace=True)

# Avaliação quantitativa de valores ausentes por coluna
missing_values = df.isnull().sum()
missing_pct = (missing_values[missing_values > 0] / len(df) * 100).round(2)
missing_summary = pd.DataFrame({
    'Valores Ausentes': missing_values[missing_values > 0],
    'Percentual (%)': missing_pct
}).sort_values(by='Percentual (%)', ascending=False)

print("\n--- Atributos com Valores Ausentes Identificados ---")
display(missing_summary)

# Avaliação e padronização da variável categórica 'gender'
print("\n--- Frequência inicial da variável 'gender' ---")
print(df['gender'].value_counts())

df['gender'] = df['gender'].replace('Unknown/Invalid', 'Não informado')
print("\n--- Frequência de 'gender' após padronização ---")
print(df['gender'].value_counts())
```

```
--- Informações Estruturais do Dataset ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 50 columns):
 #   Column                    Non-Null Count   Dtype 
---  ------                    --------------   ----- 
 0   encounter_id              101766 non-null  int64 
 1   patient_nbr               101766 non-null  int64 
 2   race                      101766 non-null  object
 3   gender                    101766 non-null  object
 4   age                       101766 non-null  object
 5   weight                    101766 non-null  object
 6   admission_type_id         101766 non-null  int64 
 7   discharge_disposition_id  101766 non-null  int64 
 8   admission_source_id       101766 non-null  int64 
 9   time_in_hospital          101766 non-null  int64 
 10  payer_code                101766 non-null  object
 11  medical_specialty         101766 non-null  object
 12  num_lab_procedures        101766 non-null  int64 
 13  num_procedures            101766 non-null  int64 
 14  num_medications           101766 non-null  int64 
 15  number_outpatient         101766 non-null  int64 
 16  number_emergency          101766 non-null  int64 
 17  number_inpatient          101766 non-null  int64 
 18  diag_1                    101766 non-null  object
 19  diag_2                    101766 non-null  object
```

```
 20  diag_3                    101766 non-null  object
 21  number_diagnoses          101766 non-null  int64
 22  max_glu_serum             101766 non-null  object
 23  A1Cresult                 101766 non-null  object
 24  metformin                 101766 non-null  object
 25  repaglinide               101766 non-null  object
 26  nateglinide               101766 non-null  object
 27  chlorpropamide            101766 non-null  object
 28  glimepiride               101766 non-null  object
 29  acetohexamide             101766 non-null  object
 30  glipizide                 101766 non-null  object
 31  glyburide                 101766 non-null  object
 32  tolbutamide               101766 non-null  object
 33  pioglitazone              101766 non-null  object
 34  rosiglitazone             101766 non-null  object
 35  acarbose                  101766 non-null  object
 36  miglitol                  101766 non-null  object
 37  troglitazone              101766 non-null  object
 38  tolazamide                101766 non-null  object
 39  examide                   101766 non-null  object
 40  citoglipton               101766 non-null  object
 41  insulin                   101766 non-null  object
 42  glyburide-metformin       101766 non-null  object
 43  glipizide-metformin       101766 non-null  object
 44  glimepiride-pioglitazone  101766 non-null  object
 45  metformin-rosiglitazone   101766 non-null  object
 46  metformin-pioglitazone    101766 non-null  object
 47  change                    101766 non-null  object
 48  diabetesMed               101766 non-null  object
 49  readmitted                101766 non-null  object
dtypes: int64(13), object(37)
memory usage: 38.8+ MB

--- Atributos com Valores Ausentes Identificados ---
```

	Valores Ausentes	Percentual (%)
weight	98569	96.86
medical_specialty	49949	49.08
payer_code	40256	39.56
race	2273	2.23
diag_3	1423	1.40
diag_2	358	0.35
diag_1	21	0.02

```
--- Frequência inicial da variável 'gender' ---
gender
Female             54708
Male               47055
Unknown/Invalid        3
Name: count, dtype: int64

--- Frequência de 'gender' após padronização ---
```

```
gender
Female           54708
Male             47055
Não informado        3
Name: count, dtype: int64
```

In [3]:
```
# 3. Avaliação e Tratamento de Registros Duplicados
duplicados_encounter = df['encounter_id'].duplicated().sum()
duplicados_linhas = df.duplicated().sum()

print(f"Total de encounter_ids duplicados: {duplicados_encounter}")
print(f"Total de linhas inteiramente duplicadas: {duplicados_linhas}")

# Remoção de duplicatas de linhas, caso existam
df.drop_duplicates(inplace=True)
print(f"Total de registros após checagem de duplicatas: {len(df)}")
```

```
Total de encounter_ids duplicados: 0
Total de linhas inteiramente duplicadas: 0

Total de registros após checagem de duplicatas: 101766
```

In [4]:
```
# 4. Remoção de Colunas Irrelevantes ou com Alta Esparsidade
# A coluna 'weight' possui mais de 96% de ausentes; 'payer_code' e
'medical_specialty' possuem volume excessivo de nulos
colunas_para_remover = [
    'weight',
    'payer_code',
    'medical_specialty'
]

df.drop(
    columns=colunas_para_remover,
    inplace=True,
    errors='ignore'
)

print(f"Total de colunas restantes após descarte: {df.shape[1]}")
print(f"Colunas descartadas que ainda constam no DataFrame: {[c for c in
colunas_para_remover if c in df.columns]}")
```

```
Total de colunas restantes após descarte: 47
Colunas descartadas que ainda constam no DataFrame: []
```

In [5]:
```
# 5. Correção de Tipos e Definição da Variável Alvo
# Conversão de identificadores para tipo textual (string)
df['encounter_id'] = df['encounter_id'].astype(str)
df['patient_nbr'] = df['patient_nbr'].astype(str)

# Criação da variável alvo binária para readmissão precoce (< 30 dias)
# 1 = Readmissão em menos de 30 dias; 0 = Readmissão em >30 dias ou sem
registro de retorno (NO)
df['readmitido_30_dias'] = df['readmitted'].map({'<30': 1, '>30': 0, 'NO':
0})

print("Tipos de dados dos identificadores e da variável alvo:")
print(df[['encounter_id', 'patient_nbr', 'readmitido_30_dias']].dtypes)

print("\nDistribuição da variável de desfecho 'readmitido_30_dias':")
distribuicao_alvo = pd.DataFrame({
    'Total': df['readmitido_30_dias'].value_counts(),
    'Percentual (%)': (df['readmitido_30_dias'].value_counts(normalize=True)
* 100).round(2)
})
display(distribuicao_alvo)
```

```
Tipos de dados dos identificadores e da variável alvo:
encounter_id          object
patient_nbr           object
readmitido_30_dias     int64
dtype: object

Distribuição da variável de desfecho 'readmitido_30_dias':
```

	Total	Percentual (%)
readmitido_30_dias		
0	90409	88.84
1	11357	11.16

In [6]:
```
# 6. Validação de Consistência e Exportação do Conjunto de Dados Resultante

# Asserções de integridade e consistência dos dados preparados
assert df['encounter_id'].is_unique, "Há IDs de atendimento repetidos!"
assert df[['encounter_id', 'patient_nbr']].notna().all().all(), "Há
identificadores nulos!"
assert not df.eq('?').any().any(), "Ainda existem caracteres '?' não
tratados no dataset!"
assert df['readmitido_30_dias'].isin([0, 1]).all(), "Há valores inválidos na
variável alvo!"
assert all(coluna not in df for coluna in colunas_para_remover), "Colunas
que deveriam ser removidas ainda constam!"

print("Todas as asserções de qualidade e consistência foram validadas com
sucesso!")
print(f"Dimensões originais:  {df_original.shape}")
print(f"Dimensões preparadas: {df.shape}")

print("\nAtributos com valores nulos remanescentes (a serem tratados na
modelagem futura):")
display(df.isna().sum()[df.isna().any()])

# Exportação do dataset resultante da etapa de preparação (RA1)
output_dir = Path("data/processed")
output_dir.mkdir(parents=True, exist_ok=True)
output_path = output_dir / "diabetic_data_clean.csv"
df.to_csv(output_path, index=False)

print(f"\nConjunto de dados resultante salvo com sucesso
em:\n{output_path.resolve()}")
```

```
Todas as asserções de qualidade e consistência foram validadas com sucesso!
Dimensões originais:  (101766, 50)
Dimensões preparadas: (101766, 48)

Atributos com valores nulos remanescentes (a serem tratados na modelagem
futura):

race      2273
diag_1      21
diag_2     358
diag_3    1423
dtype: int64

Conjunto de dados resultante salvo com sucesso em:
C:\Users\HelenLauren\develop\Data-
analytics\data\processed\diabetic_data_clean.csv
```

Resultado da Preparação dos Dados (RA1)

A base resultante consolidada possui **101.766 atendimentos** e **48 colunas**, devidamente validada e exportada para o arquivo `data/processed/diabetic_data_clean.csv`.

Decisões e Conclusões da Preparação:

- Os identificadores (`encounter_id` , `patient_nbr`) foram validados e garantidos como únicos e de tipo textual.
- Não foram detectadas duplicatas de atendimento.
- Os caracteres de ausência `?` foram totalmente tratados e padronizados.
- As colunas de alta esparsidade (`weight` , `payer_code` , `medical_specialty`) foram eliminadas para evitar distorções analíticas.
- A variável alvo binária `readmitido_30_dias` foi construída com consistência lógica (`1` para readmissão precoce `<30` , `0` para outros casos).
- Os valores ausentes remanescentes em `race` e nos diagnósticos foram preservados de maneira clínica, devendo receber tratamento apropriado nas etapas posteriores de modelagem e transformação.
- Conforme as instruções do RA1, agregações analíticas aprofundadas, criação de indicadores de performance e gráficos de conclusão foram mantidos para as próximas entregas do projeto.

RA2 - Data Stream

1. Objetivo da Análise

O objetivo desta etapa é demonstrar a aplicação dos conceitos de **processamento de dados em streaming** utilizando o **Apache Spark Structured Streaming**, no contexto do projeto de readmissão hospitalar desenvolvido no RA1.

Como a base histórica de atendimentos hospitalares é estática, foi adotado o **Cenário A**, simulando a chegada contínua de novos registros de atendimentos artificiais em diferentes momentos temporais.

A análise utiliza **janelas temporais (*tumbling windows*)** para monitorar continuamente o fluxo de atendimentos e calcular o volume de pacientes com readmissão prevista em até 30 dias após a alta hospitalar.

In [7]:
```
# 1. Configuração do ambiente e detecção do JAVA_HOME
import os
import platform

# Configuração multiplataforma: define o caminho do Java apenas caso
necessário e se o diretório existir
default_macos_java = (
    "/opt/homebrew/Cellar/openjdk@17/17.0.17/"
    "libexec/openjdk.jdk/Contents/Home"
)

if "JAVA_HOME" not in os.environ and os.path.exists(default_macos_java):
    os.environ["JAVA_HOME"] = default_macos_java
    os.environ["PATH"] = default_macos_java + "/bin:" + os.environ["PATH"]

print("Sistema Operacional:", platform.system())
print("JAVA_HOME:", os.environ.get("JAVA_HOME", "Padrão do Sistema"))
```

```
Sistema Operacional: Windows
JAVA_HOME: C:\Program Files\Java\jdk-21
```

In [8]:
```
# 2. Importação de bibliotecas para streaming
import json
import time
import shutil
from pathlib import Path

from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructType,
    StructField,
    StringType,
    IntegerType,
    TimestampType
)
from pyspark.sql.functions import (
    col,
    window,
    count,
    sum as spark_sum
)

print("Bibliotecas PySpark carregadas com sucesso.")
```

```
Bibliotecas PySpark carregadas com sucesso.
```

In [9]:
```
# 3. Preparando diretórios para recepção de fluxo de dados
base_dir = Path("streaming_demo")
input_dir = base_dir / "input"
checkpoint_dir = base_dir / "checkpoint"

# Limpar arquivos e checkpoints de execuções anteriores
if base_dir.exists():
    shutil.rmtree(base_dir)

input_dir.mkdir(parents=True, exist_ok=True)

print("Diretório de entrada de streaming:", input_dir.resolve())
print("Diretório de checkpoint:", checkpoint_dir.resolve())
```

```
Diretório de entrada de streaming: C:\Users\HelenLauren\develop\Data-
analytics\streaming_demo\input
Diretório de checkpoint: C:\Users\HelenLauren\develop\Data-
analytics\streaming_demo\checkpoint
```

In [10]:
```
# 4. Inicializando a sessão Spark com configurações adequadas
spark = (
    SparkSession.builder
    .appName("ReadmissionRatesStreaming")
    .master("local[2]")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate()
)

print("Versão do Apache Spark:", spark.version)
print("Sessão Spark iniciada com sucesso.")
```

```
Versão do Apache Spark: 4.2.0
Sessão Spark iniciada com sucesso.
```

In [11]:
```
# 5. Definição do schema estruturado para o streaming
# O campo readmitted_30_days vale 1 quando representa readmissão em até 30
dias e 0 caso contrário.

schema = StructType([
    StructField("encounter_id", StringType(), False),
    StructField("event_time", TimestampType(), False),
    StructField("time_in_hospital", IntegerType(), False),
    StructField("readmitted_30_days", IntegerType(), False)
])

print("Esquema definido com sucesso.")
```

```
Esquema definido com sucesso.
```

2. Janelas Temporais

Foi utilizada uma janela temporal fixa e não sobreposta (*tumbling window*) com duração de **10 segundos**.

- **Duração da janela:** 10 segundos (`10 seconds`).
- **Deslocamento:** Não aplicável (janela *tumbling* é contígua e sem sobreposição).
- **Operações de Agregação:**
  - `count("*")` : Total de atendimentos hospitalares recebidos no intervalo.
  - `sum("readmitted_30_days")` : Total acumulado de readmissões em até 30 dias.
- **Tratamento de atrasos (Watermark):** Configurada tolerância de atraso de 1 minuto (`withWatermark("event_time", "1 minute")`).
- **Modo de Saída:** Modo `update` , emitindo e atualizando os resultados no console à medida que novos micro-lotes são computados.

In [12]:
```
# 6. Configuração da leitura em streaming e agregação por janela temporal
stream_df = (
    spark.readStream
    .schema(schema)
    .option("maxFilesPerTrigger", 1)
    .json(str(input_dir.resolve()))
)

aggregated_df = (
    stream_df
    .withWatermark("event_time", "1 minute")
    .groupBy(
        window(col("event_time"), "10 seconds")
    )
    .agg(
        count("*").alias("total_atendimentos"),
        spark_sum("readmitted_30_days").alias("total_readmissoes_30_dias")
    )
    .select(
        col("window.start").alias("inicio_janela"),
        col("window.end").alias("fim_janela"),
        col("total_atendimentos"),
        col("total_readmissoes_30_dias")
    )
)

print("Fluxo e agregação por janelas temporais configurados.")
```

```
Fluxo e agregação por janelas temporais configurados.
```

In [13]:
```python
# 7. Iniciando a query de streaming com saída no console
query = (
    aggregated_df.writeStream
    .outputMode("update")
    .format("console")
    .option("truncate", "false")
    .option("checkpointLocation", str(checkpoint_dir.resolve()))
    .trigger(processingTime="2 seconds")
    .start()
)

print("Streaming iniciado com sucesso. Ativo:", query.isActive)
```

```
Streaming iniciado com sucesso. Ativo: True
```

In [14]:
```
# 8. Criação dos registros artificiais coerentes com o contexto do projeto
registros = [
    {
        "encounter_id": "SIM001",
        "event_time": "2026-10-14T15:00:01",
        "time_in_hospital": 4,
        "readmitted_30_days": 1
    },
    {
        "encounter_id": "SIM002",
        "event_time": "2026-10-14T15:00:05",
        "time_in_hospital": 2,
        "readmitted_30_days": 0
    },
    {
        "encounter_id": "SIM003",
        "event_time": "2026-10-14T15:00:12",
        "time_in_hospital": 6,
        "readmitted_30_days": 1
    },
    {
        "encounter_id": "SIM004",
        "event_time": "2026-10-14T15:00:16",
        "time_in_hospital": 3,
        "readmitted_30_days": 0
    },
    {
        "encounter_id": "SIM005",
        "event_time": "2026-10-14T15:00:22",
        "time_in_hospital": 5,
        "readmitted_30_days": 1
    }
]

print("Total de registros simulados a serem enviados:", len(registros))
```

```
Total de registros simulados a serem enviados: 5
```

In [15]:
```
# 9. Simulação da chegada dos registros em momentos distintos (um arquivo a cada 3 segundos)
for i, registro in enumerate(registros, start=1):
    arquivo = input_dir / f"registro_{i:03d}.json"

    with open(arquivo, "w", encoding="utf-8") as f:
        json.dump(registro, f)

    print(f"Novo registro enviado: {registro['encounter_id']} (event_time: {registro['event_time']})")
    time.sleep(3)
```

```
Novo registro enviado: SIM001 (event_time: 2026-10-14T15:00:01)

Novo registro enviado: SIM002 (event_time: 2026-10-14T15:00:05)

Novo registro enviado: SIM003 (event_time: 2026-10-14T15:00:12)

Novo registro enviado: SIM004 (event_time: 2026-10-14T15:00:16)

Novo registro enviado: SIM005 (event_time: 2026-10-14T15:00:22)
```

In [16]:
```
# 10. Processamento de todos os arquivos remanescentes e encerramento controlado
query.processAllAvailable()
time.sleep(2)

print("Streaming ativo antes do encerramento:", query.isActive)

query.stop()
try:
    query.awaitTermination(timeout=5)
except Exception:
    pass

spark.stop()
print("Sessão Spark e consulta de streaming encerradas com sucesso.")
```

```
Streaming ativo antes do encerramento: True

Sessão Spark e consulta de streaming encerradas com sucesso.
```

3. Resultados e Interpretação

Durante a execução, foram enviados cinco registros artificiais (`SIM001` a `SIM005`) em momentos cronológicos distintos. O Spark Structured Streaming processou os arquivos JSON consumidos do diretório monitorado e agrupou os atendimentos nas janelas temporais correspondentes de 10 segundos.

Os resultados consolidados observados nas janelas foram:

Janela Temporal	Total de Atendimentos	Readmissões em até 30 dias
15:00:00 – 15:00:10	2	1
15:00:10 – 15:00:20	2	1
15:00:20 – 15:00:30	1	1
Total Acumulado	**5**	**3**

Comportamento Observado:

- À medida que novos arquivos JSON foram disponibilizados na pasta monitorada, a cada micro-lote o Spark atualizou a contagem agregada da janela em vigência.
- Na primeira janela (15:00:00 a 15:00:10), a chegada do evento `SIM001` contabilizou 1 atendimento e 1 readmissão; subsequentemente, a entrada de `SIM002` incrementou o total de atendimentos para 2 sem elevar o número de readmissões (visto que `readmitted_30_days` = 0).
- Nas janelas subsequentes, observou-se idêntica consistência no processamento de eventos nas janelas temporais de 10 segundos.

4. Conclusão

A atividade demonstrou com sucesso a aplicação do Apache Spark Structured Streaming em um fluxo simulado de dados clínicos relacionados ao projeto de readmissão hospitalar.

A utilização de janelas temporais *tumbling* permitiu monitorar a taxa de chegada de atendimentos hospitalares e a identificação contínua de casos de readmissão precoce, confirmando a viabilidade de pipelines em streaming para vigilância em tempo real em ambientes de saúde.